# Homework_3

June 25, 2026

## 1 Lists []

```
[ ]: students = ['Marco', 'Elena', 'Theo', 'Priya', 'Caleb'] # Creates a list.
     students
```

```
[ ]: ['Marco', 'Elena', 'Theo', 'Priya', 'Caleb']
```

```
[ ]: students.append('Sofia') # Adds this to the end of the list.
     students # If I ran the cell again it would keep adding Sofia.
```

```
[ ]: ['Marco', 'Elena', 'Theo', 'Priya', 'Caleb', 'Sofia']
```

```
[ ]: students.remove('Theo') # Removes the name from the list.
     students # If I ran this cell again I would get an error.
```

```
[ ]: ['Marco', 'Elena', 'Priya', 'Caleb', 'Sofia']
```

```
[ ]: sorted(students) # Sorts the students names
```

```
[ ]: ['Caleb', 'Elena', 'Marco', 'Priya', 'Sofia']
```

**Code Explanation:**

First I created a list called ‘students’, then used the following methods to manipulate the data.

- Append(): This will add whatever data you put in the () to the end of your list. You have to be careful because everytime you run this command it will add that value again and you’ll end up with duplicate data.
- Remove(): This will remove the value that is inside the () from your list. Now if you run the cell and it removes the value and you do it again it will return an error because the value has been removed.
- Sorted(): This method will sort your data either by ABC’s or numerically (acending). This will depend on if your data is full of strings or numbers.

## 2 Dictionaries

[ ]:
```python
Student_dict = {
    'Marco': 91,
    'Elena': 74,
    'Theo': 83,
    'Priya': 97,
    'Caleb': 68
} # Creates a dictionary.
Student_dict
```

[ ]:
```
{'Marco': 91, 'Elena': 74, 'Theo': 83, 'Priya': 97, 'Caleb': 68}
```

[ ]:
```python
Student_dict['Sofia'] = 79 # Adds a new Sofia key and 79 value pair.
Student_dict
```

[ ]:
```
{'Marco': 91, 'Elena': 74, 'Theo': 83, 'Priya': 97, 'Caleb': 68, 'Sofia': 79}
```

[ ]:
```python
Student_dict['Caleb'] = 75 # Updates keys value.
Student_dict
```

[ ]:
```
{'Marco': 91, 'Elena': 74, 'Theo': 83, 'Priya': 97, 'Caleb': 75, 'Sofia': 79}
```

[ ]:
```python
for student in Student_dict: # Student is the key value.
    print(f'{student} {Student_dict[student]}')
```

```
Marco 91
Elena 74
Theo 83
Priya 97
Caleb 75
Sofia 79
```

[ ]:
```python
values = list(Student_dict.values()) # Places all values into a list.
values
```

[ ]:
```
[91, 74, 83, 97, 75, 79]
```

[ ]:
```python
Mean = sum(values)/len(values) # Finds the mean.
Max = max(values) # Finds max.
Min = min(values) # Finds min.
print(f'Mean grade: {Mean}')
print(f'Highest grade: {Max}')
print(f'Lowest grade: {Min}')
```

```
Mean grade: 83.16666666666667
Highest grade: 97
Lowest grade: 74
```

**Code Explanation:**

First I created a dictionary named “Student_dict” with a number of key and value pairs. Then I added a new key(Sofia) and value(79) using ‘Student_dict['Sofia'] = 79’. I updated the value for the key “Caleb” using “Student_dict[‘Caleb'] = 75”. I used a for loop to print the keys and the values. I used “student” for the item variable and used an f string to print “student” which would return the key and “Student_dict[student]” to print the values. Something I would like to note is “Student_dict[student]” was recommend by the inline suggestions, I liked the simplicity of it taking each key and showing the value for that key using the “student” variable. I created a variable “values = list(Student_dict.values())”, so I could get the values into a list. Then I used “Mean = sum(values)/len(values)”, “Max = max(values)”, and “Min = min(values)” to find the mean, the highest grade, and lowest grade respectively.

## 3 APIs

[ ]:
```python
import requests

# API endpoint for historical weather
url = "https://historical-forecast-api.open-meteo.com/v1/forecast"

# Parameters for the request
params = {
"latitude": 38.0293, # New latitude for Charlottesville, Virginia
"longitude": -78.4767, # New longitude for Charlottesville, Virginia
"start_date": "2026-03-01", # Start date for the past week
"end_date": "2026-03-07", # End date for the past week
"daily": "temperature_2m_min", # Daily min temperature
"timezone": "auto" # Automatically adjust for timezone
}

# Fetch data from the API
response = requests.get(url, params=params)
data = response.json()
data
```

[ ]:

```
{'latitude': 38.031494,
 'longitude': -78.465256,
 'generationtime_ms': 14.423131942749023,
 'utc_offset_seconds': -14400,
 'timezone': 'America/New_York',
 'timezone_abbreviation': 'GMT-4',
 'elevation': 151.0,
 'daily_units': {'time': 'iso8601', 'temperature_2m_min': '°C'},
 'daily': {'time': ['2026-03-01',
   '2026-03-02',
   '2026-03-03',
   '2026-03-04',
```

```
 '2026-03-05',
 '2026-03-06',
 '2026-03-07'],
'temperature_2m_min': [6.4, -0.7, 0.1, 6.0, 8.3, 10.6, 9.1]}}
```

[ ]:
```python
jan_2024_first = data['daily']['temperature_2m_min'] # pulls temperature list
jan_2024_first # pulls temperature list for first week in Jan 2024
```

[ ]:
```
[1.7, -0.2, -3.3, -0.7, -6.0, -2.8, -0.8]
```

[ ]:
```python
feb_2024_first = data['daily']['temperature_2m_min']
feb_2024_first # pulls temperature list for first week in Feb 2024
```

[ ]:
```
[-0.5, 4.8, -1.0, -3.4, -3.9, -3.4, -4.9]
```

[ ]:
```python
mar_2024_first = data['daily']['temperature_2m_min']
mar_2024_first
```

[ ]:
```
[-3.0, 5.5, 7.3, 6.8, 10.6, 11.4, 9.6]
```

[ ]:
```python
jan_2025_first = data['daily']['temperature_2m_min']
jan_2025_first
```

[ ]:
```
[2.3, 0.0, -1.1, -2.2, -6.3, -3.4, -4.7]
```

[ ]:
```python
feb_2025_first = data['daily']['temperature_2m_min']
feb_2025_first
```

[ ]:
```
[1.3, -4.9, -0.3, 5.2, 1.3, -0.5, 4.9]
```

[ ]:
```python
mar_2025_first = data['daily']['temperature_2m_min']
mar_2025_first
```

[ ]:
```
[2.9, -5.6, -6.9, 0.1, 11.4, 2.3, 0.4]
```

[ ]:
```python
jan_2026_first = data['daily']['temperature_2m_min']
jan_2026_first
```

[ ]:
```
[-1.5, -3.4, -3.2, -2.9, -3.9, 2.3, 7.2]
```

[ ]:
```python
feb_2026_first = data['daily']['temperature_2m_min']
feb_2026_first
```

[ ]:
```
[-9.6, -8.5, -1.9, -0.3, -4.4, -7.9, -7.4]
```

[ ]:
```python
mar_2026_first = data['daily']['temperature_2m_min']
mar_2026_first
```

[ ]: [6.4, -0.7, 0.1, 6.0, 8.3, 10.6, 9.1]

```
[ ]: # I did all of these in one cell for better readability
print("Here are the lowest temperatures:")
lowest_jan_2024_first = min(jan_2024_first) # gets lowest temp in list
lowest_feb_2024_first = min(feb_2024_first)
lowest_mar_2024_first = min(mar_2024_first)
# prints the lowest value for each month for that year
print(f'Jan 2024 {lowest_jan_2024_first} | Feb 2024 {lowest_feb_2024_first} |␣
 ↪Mar 2024 {lowest_mar_2024_first}')
# repeat the same thing just different year and month combo
lowest_jan_2025_first = min(jan_2025_first)
lowest_feb_2025_first = min(feb_2025_first)
lowest_mar_2025_first = min(mar_2025_first)
print(f'Jan 2025 {lowest_jan_2025_first} | Feb 2025 {lowest_feb_2025_first} |␣
 ↪Mar 2025 {lowest_mar_2025_first}')
lowest_jan_2026_first = min(jan_2026_first)
lowest_feb_2026_first = min(feb_2026_first)
lowest_mar_2026_first = min(mar_2026_first)
print(f'Jan 2026 {lowest_jan_2026_first} | Feb 2026 {lowest_feb_2026_first} |␣
 ↪Mar 2026 {lowest_mar_2026_first}')
```

```
Here are the lowest temperatures:
Jan 2024 -6.0 | Feb 2024 -4.9 | Mar 2024 -3.0
Jan 2025 -6.3 | Feb 2025 -4.9 | Mar 2025 -6.9
Jan 2026 -3.9 | Feb 2026 -9.6 | Mar 2026 -0.7
```

**Code Explanation:**

For this code I made a get request to the open-meteo Weather API. I had the latitude and longitude set for Charlottesville, VA in the parameters. Then I would get the lowest temperatures for the first seven days of the months of January, February, and March. I did this for the years 2024, 2025, and 2026. I would set the “start_date” and “end_date” parameters to get the data for the Month and year combo above. I would make a variable following this format “month_year_first” and set it equal to “data[‘daily’][‘temperature_2m_min’]” which would return a list of the lowest temperatures each day in the time slot. Finally I used the min() function for each list variables to find the lowest temperature in each list. Then I used three f strings to print the results out for each year and I lined them up to make it easier to read.

## 4 Split()

```
[ ]: string = 'Marco,Elena,Theo,Priya,Caleb'
string
```

[ ]: 'Marco,Elena,Theo,Priya,Caleb'

```
[ ]: string.split(',') # Splits the parts by comma and places into a list.
```

[ ]: ['Marco', 'Elena', 'Theo', 'Priya', 'Caleb']

[ ]:
```
sentence = 'Python makes it easy to work with lists and strings.'
sentence
```

[ ]: 'Python makes it easy to work with lists and strings.'

[ ]:
```
sentence.split(" ") # Splits sentence into a list of words.
```

[ ]:
```
['Python',
 'makes',
 'it',
 'easy',
 'to',
 'work',
 'with',
 'lists',
 'and',
 'strings.']
```

[ ]:
```
custom_delimiters = 'Marco|Elena|Theo|Priya|Caleb'
custom_delimiters
```

[ ]: 'Marco|Elena|Theo|Priya|Caleb'

[ ]:
```
custom_delimiters.split('|') # Splits the string by |
```

[ ]: ['Marco', 'Elena', 'Theo', 'Priya', 'Caleb']

**How Split() Works:**

The split() method is used to split a string by a certain character or space and placing each part into a list. It can be useful for seeing how many words are in a string or it can help with finding specific words in a string.

**Code Explanation:**

Here I used the split() method on 3 different strings.

1. I had "string = 'Marco,Elena,Theo,Priya,Caleb' ", I used "string. split(',')" to create a list of strings separated by the comma's in the string.
2. For "sentence = 'Python makes it easy to work with lists and strings.' " I used "sentence.split(' ')" to create a list of strings that are separated by spaces.
3. Finally for "custom_delimiters = 'Marco|Elena|Theo|Priya|Caleb' " I created used custom_delimiters.split('|') to create a list of strings separted by the '|' in the string.

**Analysis of API Data:**

After looking at the data the lowest low temperature was February of 2026 being -9.6 degrees celicus. While the highest low temperature was March of 2026 being -0.7 degrees celicus. The month of January seems to have consistent numbers except for this year with -3.9 degress celicus. Same thing with February seems that 2026 has an outlier this time being colder. March seems to have the most variation, which makes sense because that when the seasons start to change.